Universidade do Minho
Escola de Engenharia

Licenciatura em Engenharia Informática

Computação Gráfica

Grupo 23

A104619, Gonçalo da Silva Carmo
A104438, Gonçalo Filipe Duarte Barbosa
A106799, Jorge Gabriel Sá Barbosa

Primeira fase da componente prática, fevereiro de 2026

Introdução **3**
Arquitetura e estruturas base **3**
Point 3
Generator 3
Engine 4
Modelação das Formas Geométricas **4**
Plano 5
Cone 6
Esfera 8
Conclusão **9**

Introdução

O objetivo principal consistiu na conceção de duas aplicações distintas: o *Generator*, responsável por interpretar comandos e exportar coordenadas tridimensionais para ficheiros .3d, e o *Engine*, encarregue de extrair especificações de um ficheiro XML e renderizar as formas geométricas na cena gráfica.

Arquitetura e estruturas base

Point

Para otimizar a manipulação de coordenadas espaciais, foi desenhada uma classe dedicada a representar pontos no espaço tridimensional (x, y, z). A segurança e a integridade dos dados são asseguradas pela utilização de métodos getter declarados como constantes, prevenindo assim alterações indesejadas ao estado do objeto.

Generator

O Generator atua como o motor lógico da geração de geometria. Ao receber os parâmetros inseridos pelo utilizador, a aplicação identifica o modelo pretendido e delega o processamento para o ficheiro .cpp correspondente, que calcula os vértices e os escreve no ficheiro de destino.

```
324
-0.5 -0.5 0.5
-0.166667 -0.5 0.5
-0.166667 -0.166667 0.5
-0.5 -0.5 0.5
-0.166667 -0.166667 0.5
-0.5 -0.166667 0.5
-0.5 -0.166667 0.5
-0.166667 -0.166667 0.5
-0.166667 0.166667 0.5
```

Figura 1- Ficheiro gerado pelo motor generator

Na primeira linha do ficheiro resultante aparece o número de vértices, de seguida de três em três linhas é representado um triângulo, que é composto pelos três floats correspondentes

Engine

O Engine atua como o componente de visualização tridimensional da aplicação. A sua configuração inicial provém de um ficheiro XML, sendo a extração de dados realizada com recurso à biblioteca tinyXML2, recomendada pela sua leveza e eficácia. O processamento e a renderização da cena baseiam-se na seguinte estrutura lógica:

loadScene: Esta função é responsável por fazer o parsing do XML. O algoritmo percorre a árvore de nós (especificamente o caminho <world> -> <group> -> <models> -> <model>) para identificar os ficheiros que contêm as formas geométricas, chamando a rotina de vetorização para cada modelo encontrado.
vectorize: Ao aceder a um ficheiro gerado, esta função extrai na primeira linha o número total de vértices (N). De seguida, lê iterativamente as coordenadas e armazena-as como instâncias da classe Point num vetor. De forma a suportar múltiplos objetos na mesma cena, o programa utiliza um vetor de vetores (std::vector<std::vector<Point>> solids), armazenando assim os pontos de todas as formas geométricas.
renderScene: Esta função encarrega-se da representação gráfica final. Após a câmara ser posicionada via gluLookAt (convertendo as coordenadas polares para cartesianas) e os eixos XYZ serem desenhados, os vértices são passados ao OpenGL com a primitiva GL_TRIANGLES. O modo de renderização está fixo em GL_LINE (glPolygonMode), apresentando os modelos em formato wireframe (linhas) para permitir a correta perceção das malhas estruturais geradas.

Modelação das Formas Geométricas

Box

A caixa é gerada centrada na origem. A sua superfície é dividida numa grelha baseada no número de divisões escolhido, onde cada sub-quadrado é formado por dois triângulos. Para otimizar o código, criámos uma função auxiliar que desenha qualquer face reaproveitando a mesma lógica matemática.

Cálculos principais:

- **Meia dimensão:** comprimento / 2
- **Tamanho de cada sub-quadrado (passo):** comprimento / divisões
- **Total de vértices gerados:** 6 faces * 2 triângulos * 3 vértices * (divisões * divisões) = 36 * (divisões * divisões).

Lógica de Construção e Orientação: Para construir a caixa completa, a função é chamada seis vezes. Em cada face, mantém-se um eixo fixo (X, Y ou Z) e calculam-se os pontos da grelha variando os outros dois eixos.

Para garantir que o motor gráfico desenha o exterior corretamente (permitindo usar o *Backface Culling* para esconder o interior), a direção em que os triângulos são desenhados tem de ser invertida nas faces opostas:

- **Ordem Normal:** Usada nas faces da Frente (Z+), Direita (X+) e Base (Y-).
- **Ordem Invertida:** Usada nas faces de Trás (Z-), Esquerda (X-) e Topo (Y+).

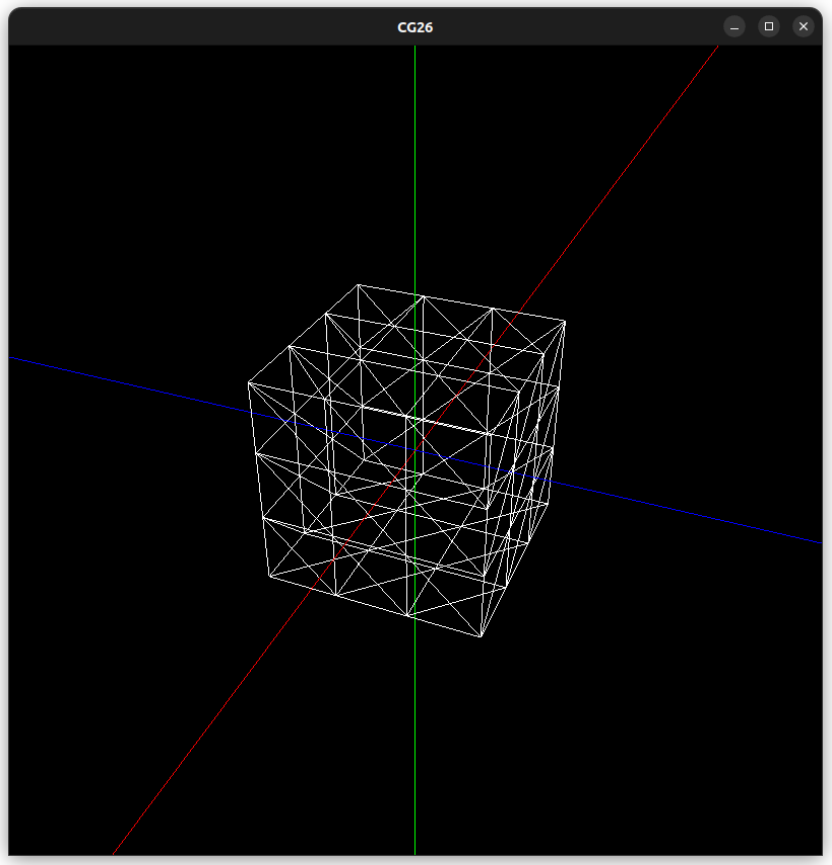

Figura 2- Box gerada pelo comando ./generator box 2 3 box_2_3.3d

---

Plano

O plano é gerado assente no eixo XZ (com a coordenada Y sempre a zero) e centrado na origem. A sua superfície é dividida numa grelha consoante o número de divisões exigido, onde cada sub-quadrado é formado por dois triângulos.

Cálculos principais:

- **Tamanho de cada sub-quadrado (passo):** comprimento / divisões
- **Total de triângulos:** 2 * (divisões * divisões)
- **Total de vértices gerados:** 3 * total de triângulos

Lógica de Construção e Orientação: Para construir o plano, o algoritmo utiliza dois ciclos de repetição que percorrem a grelha ao longo dos eixos X e Z. Em cada iteração, as

coordenadas da grelha são calculadas e é-lhes subtraída a meia dimensão (comprimento / 2) para garantir que toda a forma fica perfeitamente centrada no ponto (0, 0, 0).

Para cada sub-quadrado, os vértices são agrupados e escritos no ficheiro de forma a desenhar dois triângulos. A ordem escolhida para ligar os pontos garante que a face do plano fica a apontar para cima (no sentido positivo do eixo Y), o que permite ao motor gráfico desenhar o lado superior corretamente e esconder a face inferior através da técnica de remoção de faces ocultas (*Backface Culling*).

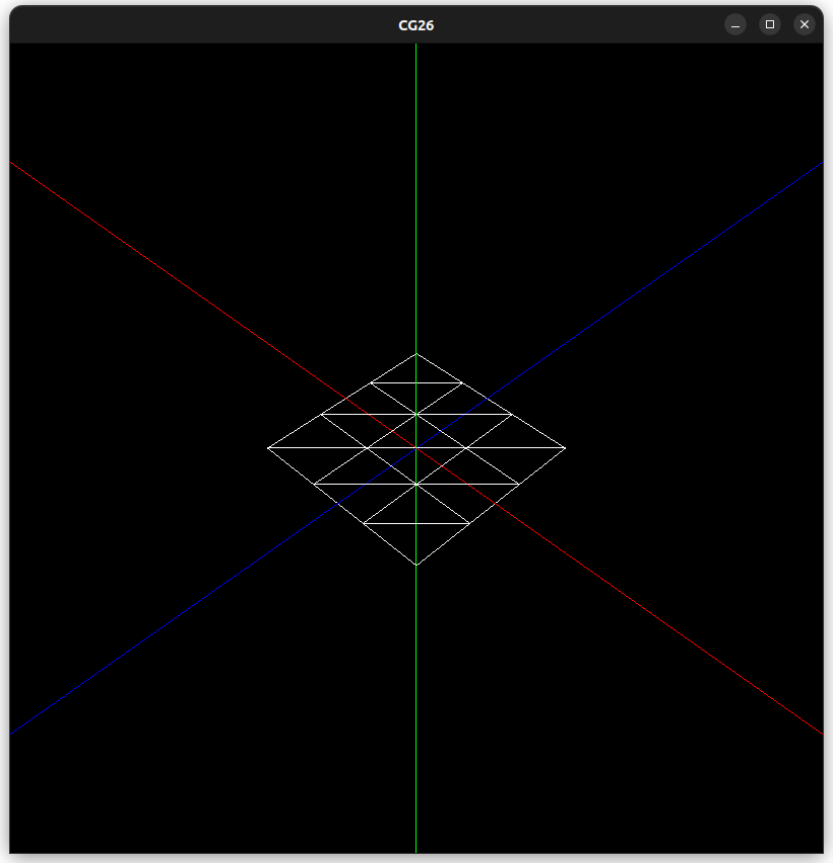

Figura 3 - Plano gerado pelo comando ./generator plane 2 3 plane_2_3.3d

---

Cone

O cone é gerado desde a sua base, assente no eixo XZ (onde Y = 0), até à sua ponta (onde Y = altura). A sua superfície é dividida em fatias à volta do eixo vertical, e em camadas horizontais desde a base até ao topo.

Cálculos principais:

- **Total de triângulos:** fatias (para a base) + (camadas - 1) * fatias * 2 (para o corpo) + fatias (para a ponta).
- **Total de vértices gerados:** total de triângulos * 3.

- **Raio do corpo:** O raio diminui de forma linear a cada camada, desde o valor máximo na base até chegar a 0 na ponta.

Lógica de Construção e Orientação: A geometria do cone obrigou a dividir o algoritmo em três fases distintas, garantindo sempre a orientação correta dos vértices para o *Backface Culling*:

- **A Base:** É fechada através de triângulos que partem do ponto central (0, 0, 0) até à borda circular. Para que a face fique a apontar para o exterior (para baixo), a ordem com que os pontos são desenhados é invertida.
- **O Corpo:** Funciona através de dois ciclos de repetição (camadas e fatias). Em cada segmento, são calculados os dois pontos do anel inferior e os dois pontos do anel superior (que tem um raio menor). Estes quatro pontos são ligados por dois triângulos desenhados na ordem normal, garantindo que as normais apontam para fora.
- **A Ponta:** A última camada do cone é um caso especial da matemática. Como o raio do anel superior é zero, desenhar dois triângulos geraria erros visuais. Assim, o algoritmo detecta quando chega à última camada e desenha apenas um triângulo por fatia, ligando os dois pontos do anel inferior diretamente ao vértice único da ponta (0, altura, 0).

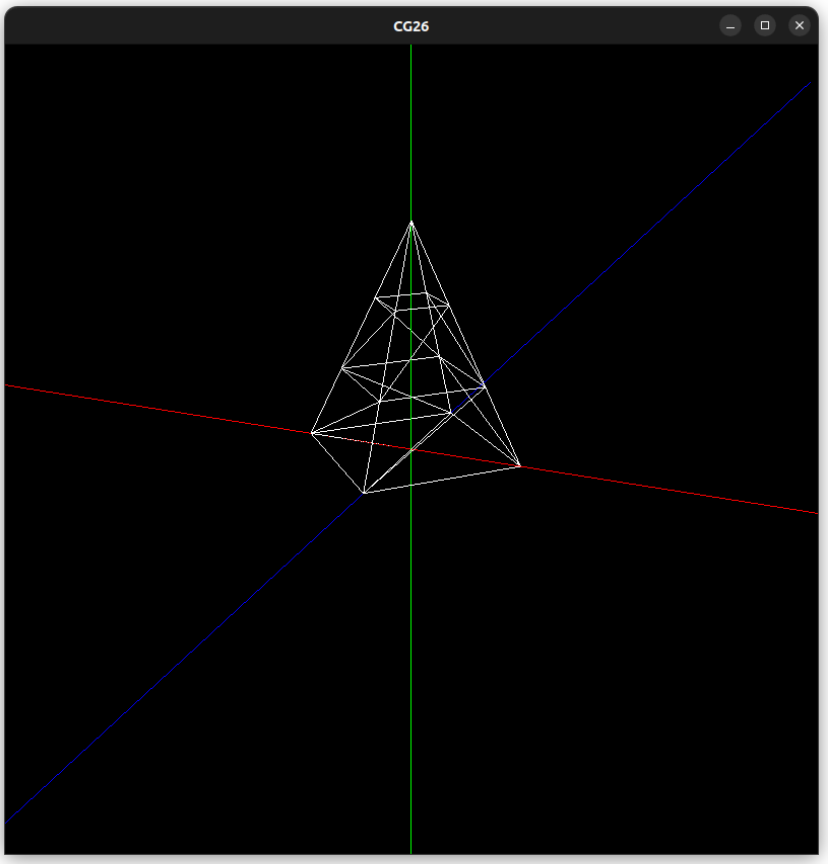

Figura 4 - Cone gerado pelo comando ./generator cone 1 2 4 3 cone_1_2_4_3.3d

Esfera

A esfera é gerada centrada na origem, recorrendo a coordenadas esféricas. A sua superfície é construída através da divisão em fatias (semelhantes aos meridianos da Terra) e camadas (semelhantes aos paralelos), com base num raio fornecido como parâmetro.

Cálculos principais:

- **Total de triângulos:** fatias (para o pólo norte) + (camadas - 2) * fatias * 2 (para o corpo) + fatias (para o pólo sul).
- **Total de vértices gerados:** total de triângulos * 3.
- **Cálculo dos pontos (X, Y, Z):** Utiliza trigonometria (senos e cossenos) para converter os ângulos de cada fatia e de cada camada em posições exatas no espaço 3D.

Lógica de Construção e Orientação: Para evitar deformações nos pólos e garantir que todas as faces apontam perfeitamente para o exterior (permitindo a utilização do *Backface Culling* no motor gráfico), o algoritmo foi dividido em três secções distintas:

- **Pólo Norte:** Corresponde à primeira camada da esfera. Como o teto é apenas um ponto exato (o topo absoluto), o algoritmo desenha apenas um triângulo por fatia, ligando esse ponto superior aos dois vértices correspondentes na primeira linha horizontal.
- **Corpo (Camadas Intermédias):** Corresponde a toda a zona central da esfera. Em cada passo, o algoritmo agrupa dois pontos da camada atual e dois pontos da camada imediatamente abaixo. Este grupo forma uma área quadrangular que é dividida em dois triângulos. Estes triângulos são sempre desenhados num sentido rigoroso (anti-horário) para que as normais apontem para fora.
- **Pólo Sul:** Corresponde à última camada. À semelhança do pólo norte, liga os dois pontos da penúltima linha horizontal diretamente ao vértice único no fundo da esfera, fechando a geometria com um triângulo por fatia.

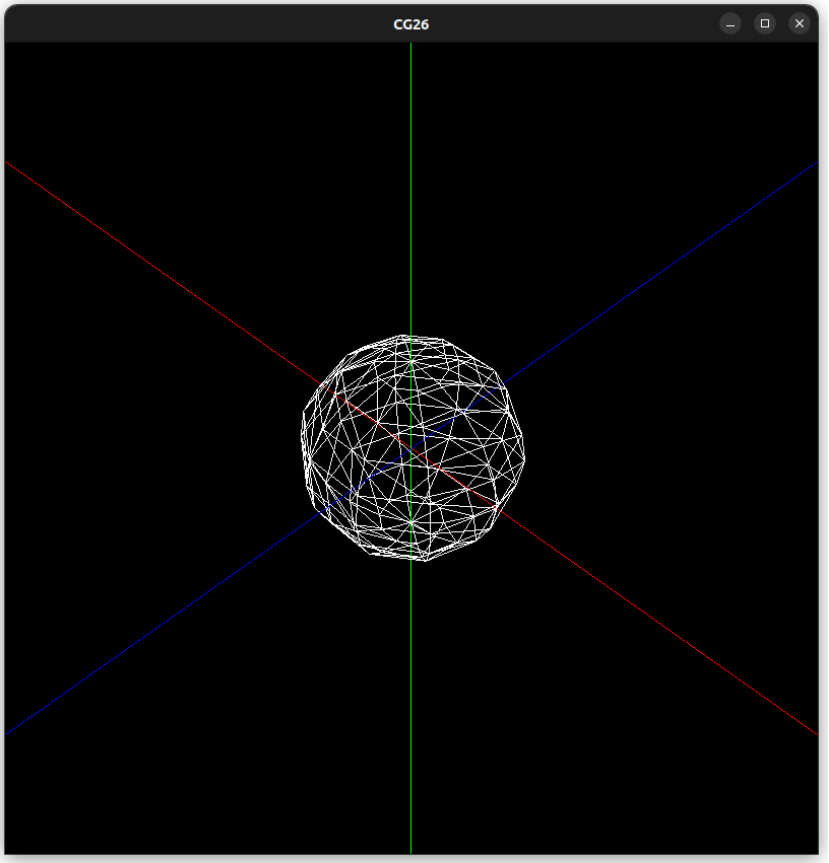

Figura 5- Esfera gerada pelo comando ./generator sphere 1 10 10 shepre_1_10_10.3d

Conclusão

O trabalho desenvolvido nesta fase inicial permitiu atingir, com sucesso, todos os objetivos propostos. A arquitetura implementada estabelece uma infraestrutura técnica sólida e estável, essencial para abordar a próxima etapa do projeto, que incidirá na implementação de transformações geométricas.